

DOCUMENTO TÉCNICO DE IMPLEMENTAÇÃO

Detecção de tópicos em feedbacks de uma Empresa Júnior usando PLN e Grafos

Divisão justa entre 5 integrantes, justificativas técnicas, contratos de implementação e análise dos resultados

Item	Definição do projeto
Entrada	Arquivo de texto feedbacks.txt, em que cada linha representa um feedback/documento textual da Empresa Júnior.
Saída principal	Comunidades de feedbacks semanticamente relacionados, interpretadas como tópicos recorrentes.
Vértice do grafo	Um feedback/documento textual.
Aresta ponderada	Relação de similaridade entre dois feedbacks, calculada por Jaccard sobre conjuntos de lemas.
Estrutura adicional	Tabela hash implementada manualmente para armazenar lemas e frequências de termos.
Algoritmos próprios	Jaccard, matriz de adjacência, Prim Máximo, remoção de arestas, DFS/BFS, métricas e análise.
Biblioteca permitida	spaCy apenas para PLN: tokenização, stopwords e lematização. O restante deve ser implementado manualmente.

Ideia central: O sistema transforma textos em conjuntos de lemas, mede a similaridade entre feedbacks, constrói um grafo ponderado, gera uma árvore geradora máxima e corta ligações fracas para revelar comunidades/tópicos.

Sumário executivo

1. Objetivo e visão geral da solução.
2. Justificativa das decisões técnicas: PLN, Jaccard, matriz, Prim Máximo e comunidades.
3. Divisão ponderada entre 5 integrantes.
4. Especificação detalhada de cada módulo, com entradas, saídas, responsabilidades e pseudocódigo.
5. Integração final, parâmetros, testes, métricas e perguntas prováveis do professor.

1. Objetivo do sistema

O objetivo é implementar um sistema de detecção de tópicos em feedbacks de uma Empresa Júnior. Cada feedback é tratado como um documento textual. O sistema cria relações entre documentos semelhantes, representa essas relações por um grafo ponderado e identifica comunidades de feedbacks. Cada comunidade é interpretada como um tópico recorrente nos dados, por exemplo: comunicação com o cliente, atrasos, qualidade técnica, preço, escopo, suporte pós-entrega ou organização da equipe.

1.1 Pipeline geral

Feedbacks

- > Lowercase
- > Remoção de acentos
- > Remoção de pontuação
- > Tokenização com spaCy
- > Remoção de stopwords
- > Lematização com spaCy

- > Conjunto de lemas
- > Similaridade de Jaccard
- > Matriz de adjacência do grafo ponderado
- > Prim Máximo
- > Remoção das menores arestas
- > DFS/BFS para componentes conexos
- > Comunidades
- > Extração de palavras frequentes
- > Nomeação dos tópicos
- > Análise dos resultados

Decisão de modelagem: O grafo será do tipo feedback-feedback. Não usaremos palavra-palavra nem grafo bipartido, porque feedback-feedback é mais direto: se dois feedbacks falam de assuntos parecidos, eles ficam conectados.

1.2 Por que essa solução atende ao enunciado?

Exigência do trabalho	Como o projeto atende
Dados textuais como entrada	Os dados são feedbacks textuais da Empresa Júnior.
Processamento de Linguagem Natural	O texto é normalizado, tokenizado, filtrado por stopwords e lematizado com spaCy.
Uso obrigatório de grafos	Cada feedback é um vértice e as arestas representam similaridade textual.
Outra estrutura de dados	Tabela hash própria, implementada pelo grupo, será usada para armazenar lemas, frequências e índices.
Algoritmos principais implementados pelo grupo	Jaccard, matriz, Prim Máximo, corte de arestas, DFS/BFS, métricas e análise serão implementados manualmente.
Análise/interpretação	Cada comunidade será nomeada como tópico usando palavras frequentes e exemplos representativos.

2. Justificativas técnicas principais

2.1 Por que usar lemas em vez de palavras cruas?

Palavras cruas podem aparecer em várias formas: "entregou", "entregando", "entregas", "entregue". A lematização reduz essas variações para uma forma base. Isso melhora o cálculo de similaridade, porque feedbacks que falam do mesmo assunto passam a compartilhar termos equivalentes.

Feedback 1: "A equipe entregou o sistema atrasado."

Feedback 2: "As entregas tiveram atrasos."

Sem lematização:

```
{equipe, entregou, sistema, atrasado}
{entregas, atrasos}
```

Com lematização:

```
{equipe, entregar, sistema, atrasar}
{entrega/entregar, atraso/atrasar}
```

Resultado: a interseção aumenta e o Jaccard fica mais fiel ao assunto.

2.2 Por que usar Jaccard?

O índice de Jaccard mede a proporção de termos em comum entre dois conjuntos. Ele é simples, interpretável e pode ser implementado manualmente sem bibliotecas externas. Como cada feedback será representado por um conjunto de lemas, Jaccard é uma escolha natural.

$$\text{Jaccard}(A, B) = |\text{intersecao}(A, B)| / |\text{uniao}(A, B)|$$

Exemplo:

A = {comunicacao, cliente, reuniao}

B = {comunicacao, prazo, reuniao}

Intersecao = {comunicacao, reuniao} -> 2 termos

Uniao = {comunicacao, cliente, reuniao, prazo} -> 4 termos

Jaccard = $2 / 4 = 0.50$

No enunciado, as arestas podem representar similaridade, coocorrência ou associação semântica. Neste projeto, a relação escolhida será similaridade textual. A associação semântica é aproximada pelo compartilhamento de lemas após o processamento linguístico.

2.3 Por que usar matriz de adjacência?

Como cada feedback será comparado com todos os outros, o grafo inicial tende a ser denso. Para N feedbacks, existem até $N(N-1)/2$ pares possíveis. Uma matriz de adjacência é adequada porque armazena diretamente o peso entre qualquer par de feedbacks e permite acesso $O(1)$ a $matriz[i][j]$.

Motivo	Explicação
Comparação todos contra todos	O cálculo de Jaccard gera uma similaridade para cada par de feedbacks.
Grafo inicialmente denso	Muitos pares podem ter alguma similaridade após a lematização.
Acesso direto ao peso	$matrix[i][j]$ retorna imediatamente a similaridade entre os feedbacks i e j.
Combina com Prim	A implementação clássica de Prim com matriz é simples e tem complexidade $O(V^2)$.
Não precisa de duas representações completas	A matriz será a representação principal. A lista de arestas será derivada apenas quando necessário para ordenar cortes e análises.

Importante: A densidade será calculada para análise, não para decidir dinamicamente entre matriz e lista. O projeto usará matriz como padrão para manter a implementação simples e coerente com o Prim Máximo.

2.4 Por que Prim Máximo?

Como o peso da aresta representa similaridade, uma árvore geradora mínima escolheria relações fracas. Isso não serve para o problema. O correto é usar uma Árvore Geradora Máxima, preservando as maiores similaridades.

Peso alto = feedbacks muito parecidos

Peso baixo = feedbacks pouco parecidos

Portanto:

Prim Mínimo -> priorizaria relações fracas -> ruim para tópicos

Prim Máximo -> prioriza relações fortes -> adequado para tópicos

Prim foi escolhido porque o grafo será armazenado em matriz de adjacência. A cada passo, o algoritmo adiciona o vértice ainda não escolhido que possui a conexão mais forte com a árvore parcial.

2.5 Por que cortar as menores arestas da árvore?

A árvore geradora máxima conecta todos os feedbacks usando as relações mais fortes possíveis, mas ainda assim ela precisa manter algumas ligações fracas para conectar grupos diferentes. Essas ligações fracas normalmente funcionam como pontes entre tópicos distintos. Ao remover as menores arestas da árvore, os grupos naturais se separam.

Exemplo conceitual:

Comunicação: F1 --0.72-- F2 --0.68-- F3

Prazos: F4 --0.81-- F5 --0.74-- F6

Ponte fraca: F3 --0.12-- F4

Ao remover F3-F4, surgem duas comunidades:

Comunidade 1: F1, F2, F3

Comunidade 2: F4, F5, F6

3. Divisão ponderada entre 5 integrantes

A divisão abaixo tenta equilibrar dificuldade técnica, responsabilidade algorítmica, documentação e capacidade de explicação na apresentação. Cada integrante possui uma parte própria, mas todas as partes têm contratos de entrada e saída para facilitar a integração.

Integrante	Módulo principal	Responsabilidades	Dificuldade estimada
1	Dados e PLN com spaCy	Entrada dos feedbacks, normalização, tokenização, stopwords, lematização e geração dos conjuntos de lemas.	Média/Alta
2	Jaccard e grafo ponderado	Implementação manual do Jaccard, matriz de adjacência, definição das arestas e métricas iniciais de grau/densidade.	Alta
3	Prim Máximo	Implementação manual da árvore geradora máxima sobre matriz, geração das arestas da AGM e explicação da escolha do Prim.	Alta
4	Corte de arestas e comunidades	Remoção das menores arestas, tratamento de vértices unitários/outliers e DFS/BFS para componentes conexos.	Alta
5	Interpretação e análise	Palavras frequentes, nomeação de tópicos, tamanhos de comunidades, cliques, relatórios de saída e interpretação dos resultados.	Média/Alta

Justiça da divisão: Os integrantes 2, 3 e 4 têm algoritmos centrais de grafos. O integrante 1 tem a parte de PLN, que é essencial para a qualidade dos pesos. O integrante 5 tem a análise, nomeação, métricas complementares e parte de apresentação dos resultados, que são critérios fortes da avaliação.

4. Integrante 1 - Dados e PLN com spaCy

4.1 Objetivo do módulo

Transformar cada feedback textual em um conjunto de lemas relevantes. Este módulo é a base de todo o sistema: se os textos forem mal processados, o Jaccard gerará pesos ruins e as comunidades ficarão incoerentes.

4.2 Entrada e saída

Formato oficial do input: arquivo TXT com um feedback por linha

O input principal do sistema será um arquivo de texto simples chamado `feedbacks.txt`, armazenado preferencialmente em `data/feedbacks.txt`. Cada linha não vazia do arquivo será interpretada como um feedback independente. Como cada feedback vira um vértice, a quantidade de linhas válidas do arquivo define a quantidade de vértices do grafo.

Exemplo de `data/feedbacks.txt`:

A comunicação com o cliente foi excelente durante todo o projeto.
O projeto atrasou e a entrega final passou do prazo.
Gostei da qualidade técnica do sistema entregue.

As reuniões de alinhamento foram confusas no começo.
O suporte pós-entrega foi rápido e atencioso.

Regras de leitura: remover linhas vazias; manter a ordem das linhas como ID dos vértices; linha 0 vira feedback F0, linha 1 vira F1 e assim por diante. Isso simplifica a matriz de adjacência, pois $matriz[i][j]$ sempre representa a similaridade entre as linhas i e j do arquivo.

Tipo	Formato sugerido	Exemplo
Entrada	Arquivo TXT: data/feedbacks.txt. Cada linha não vazia é um feedback e vira um vértice do grafo.	Linha 1: "A comunicação com o cliente foi excelente."
Saída 1	Lista de conjuntos de lemas	{comunicacao, cliente, excelente}
Saída 2	Tabela hash de frequência global	{comunicacao: 12, prazo: 9, cliente: 7}
Saída 3	Mapeamento ID -> feedback original	0 -> "A comunicação..."

4.3 Etapas de implementação

1. Carregar os feedbacks do arquivo TXT data/feedbacks.txt, considerando cada linha não vazia como um feedback.
2. Converter para lowercase.
3. Remover pontuação.
4. Usar spaCy para tokenizar.
5. Remover stopwords, espaços, números soltos e tokens muito curtos.
6. Lematizar cada token com spaCy.
7. Remover acentos dos lemas usando unicodedata da biblioteca padrão do Python.
8. Criar um conjunto de lemas para cada feedback.
9. Atualizar uma tabela hash de frequência global dos lemas.

Bibliotecas: É permitido usar spaCy para tokenização, stopwords e lematização. Para remover acentos, usar unicodedata, que faz parte da biblioteca padrão do Python. Não usar unidecode se a regra for não usar bibliotecas externas além de spaCy.

4.4 Pseudocódigo

```
função ler_feedbacks_txt(caminho):  
    feedbacks = lista vazia  
    abrir arquivo em modo leitura com codificação UTF-8  
    para cada linha do arquivo:  
        texto = linha.strip()  
        se texto não estiver vazio:  
            adicionar texto em feedbacks  
    retornar feedbacks
```

Observação: cada posição da lista retornada será o ID do vértice no grafo. `feedbacks[0]` corresponde ao vértice 0, `feedbacks[1]` ao vértice 1, e assim sucessivamente.

```
função remover_acentos(texto):  
    normalizado = unicodedata.normalize("NFD", texto)  
    retornar caracteres que não são marcas de acento
```

```
função preprocesar_feedback(texto):  
    texto = texto.lower()  
    texto = remover_pontuacao(texto)
```

```

doc_spacy = nlp(texto)
conjunto = conjunto_vazio

para cada token em doc_spacy:
    se token é stopwords: continue
    se token é pontuação ou espaço: continue
    se token possui tamanho < 3: continue

    lema = token.lemma_.lower()
    lema = remover_acentos(lema)
    lema = limpar_pontuacao(lema)

    se lema válido:
        adicionar lema ao conjunto

retornar conjunto

```

4.5 Por que usar conjunto?

O Jaccard trabalha com interseção e união de conjuntos. Por isso, cada feedback será representado como um set de lemas. Palavras repetidas dentro do mesmo feedback não aumentam o Jaccard diretamente. A frequência será usada depois, na etapa de nomeação dos tópicos.

4.6 O que o integrante 1 precisa explicar na apresentação

- PLN foi usado para transformar texto livre em uma representação comparável.
- Tokenização divide o texto em palavras/tokens.
- Stopwords são removidas porque não ajudam a identificar o assunto.
- Lematização reduz variações de palavras para melhorar a similaridade.
- A tabela hash de frequência será reutilizada na análise dos tópicos.

5. Integrante 2 - Jaccard, matriz de adjacência e grafo ponderado

5.1 Objetivo do módulo

Construir o grafo ponderado. Cada vértice representa um feedback. O peso da aresta entre dois feedbacks é a similaridade de Jaccard entre os conjuntos de lemas gerados pelo módulo de PLN.

5.2 Implementação manual do Jaccard

```

função jaccard(conjunto_A, conjunto_B):
    se A e B estiverem vazios:
        retornar 0

    intersecao = quantidade de elementos que estão em A e em B
    uniao = quantidade de elementos que estão em A ou em B

    retornar intersecao / uniao

```

Sem biblioteca pronta: Não usar sklearn, scipy, networkx ou função pronta de similaridade. A interseção e a união devem ser calculadas pelo grupo usando sets/dicionários do Python.

5.3 Matriz de adjacência ponderada

A matriz terá tamanho $N \times N$, onde N é a quantidade de feedbacks. A posição $matriz[i][j]$ guarda a similaridade entre o feedback i e o feedback j . Como o grafo é não direcionado, $matriz[i][j] = matriz[j][i]$. A diagonal principal será zero, pois um feedback não precisa se ligar a ele mesmo.

```

função construir_matriz(conjuntos_de_lemas):
    n = quantidade de feedbacks
    matriz = matriz n x n preenchida com 0

    para i de 0 até n-1:
        para j de i+1 até n-1:
            peso = jaccard(conjuntos_de_lemas[i], conjuntos_de_lemas[j])
            matriz[i][j] = peso
            matriz[j][i] = peso

    retornar matriz

```

5.4 Cálculo de grau médio e densidade

Para que a densidade e o grau médio façam sentido, é preciso definir quando uma aresta é considerada relevante. A recomendação é considerar aresta relevante quando o peso for maior ou igual a um limiar, por exemplo 0.10 ou 0.15.

$densidade = 2E / (V * (V - 1))$

grau(v) = quantidade de arestas relevantes incidentes em v
 grau_medio = soma dos graus / V

E = quantidade de arestas com peso $\geq$ LIMIAR_RELEVANCIA
 V = quantidade de vértices/feedbacks

Cuidado técnico: Se vocês considerarem todas as posições da matriz como arestas, o grafo sempre parecerá completo. Para análise, contem apenas arestas com peso acima de um limiar de relevância.

5.5 Saídas do módulo

Saída	Descrição	Quem usa depois
matriz_pesos	Matriz N x N com Jaccard entre feedbacks.	Integrante 3 e 4
arestas_relevantes	Quantidade de pares com peso acima do limiar.	Integrante 5
grau_por_vertice	Número de conexões relevantes de cada feedback.	Integrante 5
densidade_inicial	Densidade do grafo de similaridade.	Integrante 5

5.6 O que o integrante 2 precisa explicar na apresentação

- A aresta representa similaridade textual entre dois feedbacks.
- O peso é o índice de Jaccard entre conjuntos de lemas.
- A matriz foi escolhida porque todos os feedbacks são comparados com todos.
- A matriz facilita o Prim Máximo e o acesso direto aos pesos.
- Densidade e grau médio ajudam a entender se os feedbacks são muito parecidos ou muito variados.

6. Integrante 3 - Prim Máximo e Árvore Geradora Máxima

6.1 Objetivo do módulo

Gerar uma árvore geradora máxima a partir do grafo ponderado. A árvore mantém todos os vértices conectados usando as arestas de maior similaridade possível, evitando ciclos. Essa árvore será a base para separar comunidades.

6.2 Diferença entre Prim mínimo e Prim máximo

Versão	O que escolhe	Serve para este projeto?
Prim mínimo	A menor aresta disponível em cada etapa.	Não, porque escolheria feedbacks pouco semelhantes.

Prim máximo	A maior aresta disponível em cada etapa.	Sim, porque preserva conexões semânticas fortes.
-------------	--	--

6.3 Estruturas internas do Prim Máximo

Estrutura	Função
in_mst	Indica se o vértice já entrou na árvore.
key	Guarda o maior peso conhecido para conectar cada vértice à árvore.
parent	Guarda o vértice anterior responsável por conectar cada vértice à árvore.
mst_edges	Lista final de arestas da árvore geradora máxima.

6.4 Pseudocódigo do Prim Máximo

```

função prim_maximo(matriz):
    n = quantidade de vértices
    in_mst = [False] * n
    key = [-infinito] * n
    parent = [-1] * n

    key[0] = 0

    repetir n vezes:
        u = vértice fora da árvore com maior key[u]
        marcar u como dentro da árvore

        para v de 0 até n-1:
            peso = matriz[u][v]
            se v está fora da árvore e peso > key[v]:
                key[v] = peso
                parent[v] = u

    mst_edges = lista vazia
    para v de 1 até n-1:
        u = parent[v]
        adicionar (u, v, matriz[u][v]) em mst_edges

    retornar mst_edges

```

6.5 Complexidade

Usando matriz de adjacência e busca linear do maior key, a complexidade é $O(V^2)$. Para o tamanho esperado do trabalho, isso é aceitável e fácil de explicar. A simplicidade é uma vantagem, pois o professor exigiu implementação própria.

6.6 Tratamento de pesos zero

Se um feedback não tiver termos em comum com os demais, as conexões dele podem ter peso zero. O Prim ainda pode conectá-lo para formar uma árvore geradora, mas essa conexão não deve ser interpretada como relação semântica forte. Na etapa seguinte, arestas muito fracas podem ser removidas e o feedback pode ser classificado como outlier ou anexado à comunidade mais próxima.

6.7 Saídas do módulo

Saída	Descrição	Quem usa depois
mst_edges	Lista de arestas da árvore: (origem, destino, peso).	Integrante 4 e 5
mst_matrix ou mst_adj	Representação da árvore para DFS/BFS.	Integrante 4
pesos_da_arvore	Pesos usados para decidir cortes.	Integrante 4 e 5

6.8 O que o integrante 3 precisa explicar na apresentação

- A árvore geradora máxima mantém todos os feedbacks conectados pelas relações mais fortes.
- Foi usado Prim Máximo porque o grafo está em matriz e os pesos representam similaridade.
- A árvore reduz o ruído do grafo original, mantendo uma estrutura conectada e interpretável.
- A complexidade $O(V^2)$ é aceitável para a quantidade esperada de feedbacks.

7. Integrante 4 - Remoção de arestas, vértices unitários e DFS/BFS

7.1 Objetivo do módulo

Separar a árvore geradora máxima em comunidades. Para isso, removem-se arestas fracas, que normalmente funcionam como pontes artificiais entre tópicos diferentes. Depois, DFS ou BFS identifica os componentes conexos resultantes.

7.2 Critério principal de remoção

A estratégia recomendada é ordenar as arestas da árvore pelo peso crescente e remover as mais fracas quando estiverem abaixo de um limiar de corte. O limiar pode ser fixo, por exemplo 0.15 ou 0.20, ou definido por experimentação.

```
Para cada aresta (u, v, peso) da árvore, em ordem crescente:
    se peso < LIMIAR_CORTE:
        remover aresta
    senão:
        manter aresta
```

Alternativa controlada: Também é possível remover as K menores arestas para gerar K+1 comunidades. Essa opção é útil em testes, mas o limiar por peso é mais fácil de justificar semanticamente.

7.3 Tratamento de vértices isolados/outliers

Após remover arestas, pode surgir uma comunidade com apenas um vértice. Isso não deve ser ignorado. O sistema precisa decidir se esse feedback deve ser anexado a uma comunidade existente ou classificado como outlier.

```
Para cada comunidade com apenas 1 vértice v:
    melhor_comunidade = nenhuma
    melhor_peso = 0
    melhor_vertice = nenhum

para cada outra comunidade C:
    para cada vértice x em C:
        peso = matriz_original[v][x]
        se peso > melhor_peso:
            melhor_peso = peso
            melhor_comunidade = C
            melhor_vertice = x

se melhor_peso >= LIMIAR_ANEXACAO:
    anexar v à melhor_comunidade
    opcional: adicionar aresta (v, melhor_vertice) no grafo final
senão:
    classificar v como outlier
```

Situação	Ação
Vértice isolado tem similaridade suficiente com alguma comunidade	Anexar à comunidade mais próxima.
Vértice isolado tem similaridade baixa com todas as comunidades	Classificar como outlier.
Grupo unitário não deve aparecer na saída principal	Exibir em seção separada: feedbacks isolados ou casos atípicos.

Por que isso é correto?: Forçar todo feedback isolado a entrar em algum tópico pode poluir os resultados. Classificar como outlier é uma interpretação válida: aquele feedback não se parece suficientemente com nenhum grupo.

7.4 DFS ou BFS: qual usar?

DFS e BFS encontram os mesmos componentes conexos em um grafo não direcionado. Portanto, para detectar comunidades, não é necessário usar os dois como algoritmos principais. A recomendação é implementar DFS iterativo como método oficial e BFS como função auxiliar de validação ou comparação.

Algoritmo	Vantagem	Risco
DFS	Simples para encontrar componentes; pode ser implementado com pilha.	Recursivo pode estourar pilha em grafos muito profundos; por isso usar versão iterativa.
BFS	Bom para expansão por níveis; usa fila.	Pode consumir mais memória em grafos muito largos.

7.5 Pseudocódigo DFS para comunidades

```
função dfs_componentes(grafo_final):  
    visitado = conjunto vazio  
    comunidades = lista vazia  
  
    para cada vértice v:  
        se v não foi visitado:  
            componente = lista vazia  
            pilha = [v]  
            marcar v como visitado  
  
            enquanto pilha não estiver vazia:  
                atual = remover topo da pilha  
                adicionar atual em componente  
  
                para cada vizinho de atual:  
                    se vizinho não foi visitado:  
                        marcar vizinho  
                        empilhar vizinho  
  
            adicionar componente em comunidades  
  
    retornar comunidades
```

7.6 Pseudocódigo BFS opcional

```
função bfs_componentes(grafo_final):  
    visitado = conjunto vazio  
    comunidades = lista vazia  
  
    para cada vértice v:  
        se v não foi visitado:  
            componente = lista vazia  
            fila = [v]  
            marcar v como visitado  
  
            enquanto fila não estiver vazia:  
                atual = remover início da fila  
                adicionar atual em componente  
  
                para cada vizinho de atual:  
                    se vizinho não foi visitado:  
                        marcar vizinho  
                        colocar vizinho no fim da fila
```

adicionar componente em comunidades

retornar comunidades

7.7 Saídas do módulo

Saída	Descrição	Quem usa depois
comunidades	Lista de grupos de IDs de feedbacks.	Integrante 5
outliers	Lista de feedbacks que não se encaixaram em nenhum grupo.	Integrante 5
grafo_final	Árvore após cortes e anexações.	Integrante 5
arestas_removidas	Arestas fracas removidas, com pesos.	Apresentação/análise

7.8 O que o integrante 4 precisa explicar na apresentação

- A remoção das menores arestas separa tópicos que estavam ligados por relações fracas.
- DFS/BFS identifica componentes conexos, e cada componente vira uma comunidade.
- Vértices isolados são anexados à comunidade mais próxima se houver similaridade suficiente; caso contrário, são outliers.
- DFS foi escolhido como principal porque é simples e adequado para componentes conexos.

8. Integrante 5 - Interpretação, nomeação de tópicos e análise dos resultados

8.1 Objetivo do módulo

Transformar comunidades numéricas em resultados interpretáveis. O sistema não deve apenas dizer que encontrou grupos; ele precisa explicar o que cada grupo significa e quais padrões foram encontrados nos feedbacks.

8.2 Extração de palavras mais frequentes

Para cada comunidade, juntar os lemas de todos os feedbacks pertencentes a ela e contar a frequência de cada termo usando uma tabela hash. As palavras mais frequentes ajudam a nomear o tópico.

```
função palavras_frequentes(comunidade, conjuntos_de_lemas):  
    freq = HashTable()  
  
    para cada id_feedback em comunidade:  
        para cada lema em conjuntos_de_lemas[id_feedback]:  
            freq.incrementar(lema)  
  
    ordenar termos por frequência decrescente  
    retornar top_k termos
```

8.3 Nomeação dos tópicos

A nomeação pode ser feita de forma interpretativa, usando as palavras mais frequentes e exemplos de feedbacks. Isso deve ser feito de maneira transparente: o sistema apresenta as palavras mais frequentes e o grupo atribui um rótulo ao tópico.

Palavras frequentes	Nome provável do tópico
prazo, atraso, entrega, cronograma	Atrasos e gestão de prazos
comunicacao, cliente, reuniao, alinhamento	Comunicação e alinhamento com o cliente
qualidade, sistema, funcionalidade, entrega	Qualidade técnica da solução
preco, orcamento, valor, custo	Preço e orçamento
suporte, manutencao, pos, retorno	Suporte pós-entrega

Uso de LLM: Caso o grupo use LLM, ela deve ser citada no slide obrigatório. A LLM pode ajudar a gerar dados fictícios, revisar explicações ou sugerir nomes de tópicos, mas a detecção das comunidades no código deve ser feita pelos algoritmos implementados pelo grupo.

8.4 Métricas de análise

Métrica	Onde calcular	Interpretação
Número de comunidades	Após DFS/BFS	Mostra quantos tópicos principais foram detectados.
Tamanho das comunidades	Após DFS/BFS	Mostra quais tópicos aparecem com mais frequência.
Grau médio	No grafo de similaridade relevante	Indica se os feedbacks tendem a se conectar bastante.
Densidade	No grafo de similaridade relevante	Indica se os temas são concentrados ou variados.
Cliques	No grafo filtrado antes da árvore	Indicam subgrupos de feedbacks fortemente interligados.
Outliers	Após tratamento de vértices unitários	Feedbacks muito diferentes do restante da coleção.

Cliques e árvore: Não faz sentido procurar cliques grandes na árvore geradora final, porque árvores não possuem ciclos. A busca de cliques deve ocorrer no grafo de similaridade filtrado antes do Prim, usando as arestas com peso acima de um limiar.

8.5 Detecção simples de cliques

A busca de clique máximo é um problema complexo. Para o trabalho, basta detectar cliques pequenos, principalmente triângulos (cliques de tamanho 3), como evidência de subtemas fortes dentro do grafo de similaridade.

```
função detectar_triangulos(matriz, limiar):
    triangulos = lista vazia
    n = tamanho da matriz

    para i de 0 até n-1:
        para j de i+1 até n-1:
            para k de j+1 até n-1:
                se matriz[i][j] >= limiar e matriz[i][k] >= limiar e matriz[j][k] >= limiar:
                    adicionar (i, j, k) em triangulos

    retornar triangulos
```

8.6 Saídas finais

Saída	Conteúdo
Tabela de comunidades	ID da comunidade, feedbacks pertencentes, tamanho e tópico nomeado.
Tabela de palavras-chave	Top termos de cada comunidade.
Lista de outliers	Feedbacks isolados e justificativa de baixa similaridade.
Resumo estatístico	Quantidade de comunidades, densidade, grau médio, cliques e tamanhos.
Exemplos de entrada e saída	Casos usados nos slides e no README do GitHub.

8.7 O que o integrante 5 precisa explicar na apresentação

- A comunidade só vira tópico após interpretação das palavras frequentes.
- Os tópicos encontrados ajudam a Empresa Júnior a identificar gargalos e pontos fortes.
- Densidade, grau médio, tamanho das comunidades e outliers ajudam na análise dos resultados.
- Cliques pequenos indicam subgrupos com alta consistência semântica.
- A LLM, se usada, não substitui os algoritmos; ela apenas apoia dados, explicações ou nomes.

9. Contratos de integração entre os módulos

Para evitar confusão entre os integrantes, cada módulo deve entregar objetos com nomes e formatos combinados. A integração fica mais simples se todos seguirem os contratos abaixo.

Objeto	Tipo sugerido	Produzido por	Consumido por
feedbacks_originais	list[str]	Integrante 1	Todos
conjuntos_lemas	list[set[str]]	Integrante 1	Integrante 2 e 5
freq_global	HashTable manual [termo -> frequência]	Integrante 1	Integrante 5
matriz_pesos	list[list[float]]	Integrante 2	Integrante 3, 4 e 5
mst_edges	list[tuple[int, int, float]]	Integrante 3	Integrante 4 e 5
grafo_final	lista/matriz final implementada pelo grupo	Integrante 4	Integrante 5
comunidades	list[list[int]]	Integrante 4	Integrante 5
outliers	list[int]	Integrante 4	Integrante 5
relatorio_resultados	arquivo txt/json gerado pelo grupo	Integrante 5	Slides/README

9.1 Estrutura sugerida de arquivos

```
projeto-pln-grafos/  
  data/  
    feedbacks.txt  
  src/  
    preprocessing.py      # Integrante 1  
    similarity_graph.py   # Integrante 2  
    prim_max.py           # Integrante 3  
    communities.py        # Integrante 4  
    analysis.py           # Integrante 5  
  main.py  
  README.md  
  requirements.txt
```

9.2 Assinaturas de funções sugeridas

```
# preprocessing.py  
read_feedbacks_txt(path: str) -> list[str]  
preprocess_feedbacks(feedbacks: list[str]) -> tuple[list[set[str]], HashTable]  
  
# similarity_graph.py  
jaccard(a: set[str], b: set[str]) -> float  
build_adjacency_matrix(sets: list[set[str]]) -> list[list[float]]  
calculate_density(matrix: list[list[float]], threshold: float) -> float  
calculate_average_degree(matrix: list[list[float]], threshold: float) -> float  
  
# prim_max.py  
prim_maximum_spanning_tree(matrix: list[list[float]]) -> list[tuple[int, int, float]]  
  
# communities.py  
build_graph_from_edges(edges: list[tuple[int, int, float]], n: int) -> dict[int, list[int]]  
remove_weak_edges(edges, matrix, cut_threshold, attach_threshold) -> tuple[dict, list[list[int]], list[int]]  
dfs_components(graph: dict[int, list[int]], n: int) -> list[list[int]]  
bfs_components(graph: dict[int, list[int]], n: int) -> list[list[int]]  
  
# analysis.py  
extract_top_terms(communities, sets, top_k: int) -> dict[int, list[tuple[str, int]]]  
name_topics(top_terms) -> dict[int, str]  
detect_triangles(matrix, threshold) -> list[tuple[int, int, int]]  
generate_report(...) -> None
```

10. Parâmetros recomendados para testes

Parâmetro	Valor inicial sugerido	Função
LIMIAR_RELEVANCIA	0.10 ou 0.15	Define quais arestas contam para densidade, grau e cliques.
LIMIAR_CORTE	0.15 ou 0.20	Define quais arestas fracas da árvore serão removidas.
LIMIAR_ANEXACAO	0.20	Define quando um vértice isolado pode entrar na comunidade mais próxima.
MIN_TAMANHO_COMUNIDADE	2	Comunidade com 1 vértice será tratada como possível outlier.
TOP_K_TERMOS	5	Quantidade de palavras principais por tópico.

Como testar limiares: Executem o sistema com dois ou três valores de corte e comparem o número de comunidades. Se gerar muitos outliers, o limiar está alto. Se gerar apenas uma comunidade gigante, o limiar está baixo.

11. Exemplo pequeno de funcionamento

ID	Feedback original	Conjunto de lemas esperado
F0	A comunicação com o cliente foi clara e constante.	{comunicacao, cliente, claro, constante}
F1	Gostei das reuniões e do alinhamento com a equipe.	{gostar, reuniao, alinhamento, equipe}
F2	A entrega atrasou e o cronograma não foi cumprido.	{entrega, atrasar, cronograma, cumprir}
F3	O projeto teve atraso na entrega final.	{projeto, atraso, entrega, final}
F4	O sistema ficou funcional e com boa qualidade técnica.	{sistema, funcional, qualidade, tecnico}
F5	A solução entregue atendeu bem às necessidades.	{solucao, entregar, atender, necessidade}

Resultado esperado após Jaccard e comunidades:

Comunidade 1 - Comunicação e alinhamento

F0, F1

Comunidade 2 - Atraso e entrega

F2, F3

Comunidade 3 - Qualidade da solução

F4, F5

Possíveis outliers:

feedbacks com baixa similaridade com todas as comunidades.

12. Checklist de entrega

- Código no GitHub atualizado até o prazo definido pelo professor.
- Nenhuma biblioteca de grafos ou similaridade pronta foi usada.
- spaCy foi usado apenas para PLN.
- README explica como executar o projeto.
- Exemplos de entrada e saída estão no repositório.
- Slides incluem: problema, modelagem do grafo, solução, implementação, exemplos, análise e uso de LLM.
- Cada integrante consegue explicar sua própria parte e como ela se conecta às outras.
- A apresentação mostra pelo menos uma comunidade real com palavras frequentes e nome do tópico.

13. Perguntas prováveis do professor e respostas sugeridas

Pergunta	Resposta sugerida
Por que cada feedback é um vértice?	Porque o objetivo é agrupar documentos/feedbacks semanticamente semelhantes. Assim, cada comunidade representa um tópico.
Por que Jaccard?	Porque os feedbacks são representados como conjuntos de lemas; Jaccard mede a proporção de termos compartilhados e é simples de implementar manualmente.
Por que matriz de adjacência?	Porque cada feedback é comparado com todos os outros, gerando um grafo inicialmente denso. A matriz também combina com Prim.
Por que Prim Máximo?	Porque pesos maiores significam maior similaridade. A versão máxima preserva as relações mais fortes.
Por que remover arestas?	Porque as menores arestas da árvore tendem a ser pontes fracas entre tópicos diferentes. Ao removê-las, surgem comunidades.
Por que DFS/BFS?	Porque depois do corte, as comunidades são componentes conexos. DFS/BFS encontra esses componentes.
E se sobrar um vértice sozinho?	Ele é comparado com as comunidades usando a matriz original. Se houver similaridade suficiente, é anexado; caso contrário, é classificado como outlier.
Por que não Dijkstra/Bellman-Ford?	O problema não é menor caminho; é detecção de comunidades por similaridade. Usá-los seria menos coerente.
Por que cliques não são analisados na árvore final?	Porque árvores não possuem ciclos. Cliques devem ser avaliados no grafo de similaridade filtrado antes da árvore.

14. Fluxo final do main.py

1. Ler feedbacks do arquivo TXT data/feedbacks.txt, um feedback por linha
2. Processar feedbacks com spaCy
3. Gerar conjuntos de lemas
4. Calcular matriz de Jaccard
5. Calcular grau médio e densidade do grafo de similaridade
6. Executar Prim Máximo
7. Remover arestas fracas da árvore
8. Executar DFS para encontrar comunidades
9. Tratar comunidades unitárias: anexar ou marcar como outlier
10. Extrair palavras frequentes de cada comunidade
11. Nomear tópicos
12. Detectar cliques no grafo de similaridade filtrado
13. Gerar relatório final e exemplos para os slides

Conclusão técnica: A solução combina PLN para transformar texto em dados estruturados, Jaccard para pesar relações, matriz de adjacência para representar o grafo, Prim Máximo para preservar conexões fortes, corte de arestas para separar grupos, DFS/BFS para detectar comunidades e análise textual para nomear os tópicos.

15. Estrutura de dados auxiliar: Tabela Hash implementada manualmente

Como a regra do trabalho exige que a estrutura de dados adicional seja implementada manualmente, não será suficiente usar apenas o dict nativo do Python como solução principal. O projeto deve conter uma classe própria de Tabela Hash, com função hash, buckets, tratamento de colisão, inserção, busca e contagem de frequências.

15.1 Onde a Tabela Hash entra no pipeline

A Tabela Hash entra principalmente após o PLN e após a detecção das comunidades. Após a lematização, os lemas relevantes de cada feedback são inseridos na estrutura para contagem de frequência. Depois que DFS/BFS encontra as comunidades, a mesma estrutura é usada para contar quais termos aparecem com mais frequência em cada comunidade. Esses termos serão usados para extrair palavras-chave e nomear tópicos.

Fluxo conceitual:

Feedbacks -> spaCy -> lemas -> HashTable de frequência -> palavras-chave -> nomeação dos tópicos

15.2 Por que não basta usar apenas dict?

O dict do Python já é implementado internamente como uma tabela hash. Porém, como o professor pediu implementação manual das estruturas e algoritmos, usar somente dict pode ser interpretado como uso de estrutura pronta da linguagem. Por isso, o projeto deve implementar uma classe própria chamada, por exemplo, HashTable.

15.3 Funções obrigatórias da HashTable

- `hash_function(chave)`: calcula a posição inicial da chave na tabela.
- `insert(chave)`: insere uma palavra com frequência inicial 1.
- `increment(chave)`: aumenta a frequência da palavra se ela já existir; caso contrário, insere.
- `get(chave)`: retorna a frequência associada à palavra.
- `items()`: retorna todos os pares palavra/frequência para ordenação e análise.
- `contains(chave)`: verifica se uma palavra existe na tabela.

15.4 Tratamento de colisões

Como duas palavras diferentes podem gerar o mesmo índice na função hash, a implementação deve tratar colisões. A estratégia recomendada é encadeamento separado: cada posição da tabela possui uma lista de pares [chave, valor]. Se duas palavras caírem no mesmo índice, elas ficam no mesmo bucket, mas em posições diferentes da lista.

15.5 Pseudocódigo da Tabela Hash

```
classe HashTable:
    iniciar(tamanho):
        self.tamanho = tamanho
        self.tabela = lista de buckets vazios

    funcao_hash(chave):
        soma = 0
        para cada caractere em chave:
            soma = soma + codigo_ascii(caractere)
        retornar soma % self.tamanho

    incrementar(chave):
        indice = funcao_hash(chave)
        bucket = self.tabela[indice]
        para cada par em bucket:
            se par.chave == chave:
                par.valor = par.valor + 1
            retornar
        adicionar [chave, 1] ao bucket
```



```

buscar(chave):
    indice = funcao_hash(chave)
    bucket = self.tabela[indice]
    para cada par em bucket:
        se par.chave == chave:
            retornar par.valor
    retornar 0

itens():
    resultado = lista vazia
    para cada bucket na tabela:
        para cada par no bucket:
            adicionar par ao resultado
    retornar resultado

```

15.6 Uso da HashTable na nomeação dos tópicos

Para cada comunidade encontrada, o sistema cria uma nova HashTable. Em seguida, percorre todos os feedbacks da comunidade e incrementa a frequência de cada lema. Ao final, os pares palavra/frequência são ordenados manualmente para obter os termos mais frequentes.

```

funcao extrair_palavras_frequentes(comunidade, conjuntos_lemas):
    tabela = HashTable()
    para cada id_feedback em comunidade:
        para cada lema em conjuntos_lemas[id_feedback]:
            tabela.incrementar(lema)
    pares = tabela.itens()
    ordenar pares por frequencia decrescente
    retornar top_k pares

```

15.7 Ajuste na divisão dos integrantes

A Tabela Hash manual deve ficar sob responsabilidade do Integrante 1 ou do Integrante 5. A recomendação é deixar com o Integrante 5, porque ela será usada diretamente na extração de palavras frequentes e nomeação dos tópicos. Assim, a divisão fica mais equilibrada: o Integrante 1 cuida do PLN; o Integrante 5 implementa a HashTable e usa essa estrutura na análise final.

15.8 Como explicar para o professor

A estrutura de dados adicional do projeto é uma Tabela Hash implementada manualmente. Ela é usada para contagem de frequência dos lemas dentro das comunidades. Essa contagem permite extrair palavras-chave e nomear os tópicos detectados pelo grafo. Portanto, a hash não é apenas uma estrutura auxiliar genérica: ela participa da etapa de interpretação dos resultados.